

PRACTICAL - 5

❖ PROBLEM – I

- **AIM :** Shape areas: an abstract Shape with abstract area(); subclasses Circle, Rectangle,
- Triangle. Put a mix in a Shape[] and loop once to print each area and the running total,
- tracking the largest — one loop handling every shape through polymorphism.

CODE :

```
1  abstract class Shape
2  {
3      abstract double area();
4  }
5  class Circle extends Shape
6  {
7      double r;
8      Circle(double r)
9      {
10         this.r=r;
11     }
12     double area()
13     {
14         return 3.14*r*r;
15     }
16 }
17 class Rectangle extends Shape
18 {
19     double l,b;
20     Rectangle(double l,double b)
21     {
22         this.l=l;
23         this.b=b;
24     }
25     double area()
26     {
27         return l*b;
28     }
29 }
30 class Triangle extends Shape
31 {
32     double b, h;
33     Triangle(double b,double h)
34     {
35         this.b=b;
36         this.h=h;
37     }
38     double area()
39     {
40         return 0.5*b*h;
41     }
42 }
43 public class Shapepr
```

```
{
    Run | Debug
    public static void main(String[] args)
    {
        Shape s[]={
            new Circle(r: 5),
            new Rectangle(l: 4, b: 6),
            new Triangle(b: 8, h: 5)
        };
        double total=0;
        double largest=0;
        for(Shape x:s)
        {
            double a=x.area();
            System.out.println("Area = "+a);
            total+=a;
            if(a>largest)
                largest=a;
        }
        System.out.println("Total Area = "+total);
        System.out.println("Largest Area = "+largest);
    }
}
```

OUTPUT SCREENSHOT :

```
Area = 78.5
Area = 24.0
Area = 20.0
Total Area = 122.5
Largest Area = 78.5
PS C:\Users\TIRTH V PATEL\D
```

LEARNING OUTCOMES :

- Understand and implement **abstract classes and abstract methods**.
- Apply **inheritance** using Circle, Rectangle, and Triangle.
- Demonstrate **runtime polymorphism** using a Shape[] array.
- Implement **method overriding** through the area() method.
- Calculate **individual areas, total area, and largest area** using a single loop.

❖ PROBLEM – II

- **AIM :** Payroll: an abstract Employee with abstract monthlySalary(); FullTime (fixed), PartTime (hours × rate) and Intern (stipend), each calling super for the

shared name/id. Loop a mixed array to print each salary and the total; use instanceof to add a note only for interns.

CODE :

```
1  abstract class Employee
2  {
3      String name;
4      int id;
5      Employee(String name,int id)
6      {
7          this.name=name;
8          this.id=id;
9      }
10     abstract double monthlySalary();
11 }
12 class FullTime extends Employee
13 {
14     double salary;
15     FullTime(String name,int id,double salary)
16     {
17         super(name,id);
18         this.salary=salary;
19     }
20     double monthlySalary()
21     {
22         return salary;
23     }
24 }
25 class PartTime extends Employee
26 {
27     int hours;
28     double rate;
29     PartTime(String name,int id,int hours,double rate)
30     {
31         super(name,id);
32         this.hours=hours;
33         this.rate=rate;
34     }
35     double monthlySalary()
36     {
37         return hours*rate;
38     }
39 }
40 class Intern extends Employee
41 {
42     double stipend;
43     Intern(String name,int id,double stipend)
```

```
{
    super(name, id);
    this.stipend=stipend;
}

double monthlySalary()
{
    return stipend;
}
}

public class Payroll
{
    Run | Debug
    public static void main(String[] args)
    {
        Employee[] employees={
            new FullTime(name: "Tirth",id: 101,salary: 25300),
            new PartTime(name: "Rahul",id: 102,hours: 80,rate: 320),
            new Intern(name: "Amit",id: 103,stipend: 10000)
        };
        double total=0;
        for(Employee e:employees)
        {
            double salary = e.monthlySalary();
            System.out.println(e.name+" Salary: "+salary);
            if(e instanceof Intern)
            {
                System.out.println(x: "Note: This employee is an Intern.");
            }
            total+=salary;
        }
        System.out.println("Total Salary: "+total);
    }
}
```

OUTPUT SCREENSHOT :

```
Tirth Salary: 25300.0
Rahul Salary: 25600.0
Amit Salary: 10000.0
Note: This employee is an Intern.
Total Salary: 60900.0
PS C:\Users\TIRTH V PATEL\Desktop
```

LEARNING OUTCOMES :

- Understand **abstract classes and inheritance** in a payroll system.
- Implement different salary calculations using **method overriding**.
- Apply **polymorphism** to process FullTime, PartTime, and Intern objects.
- Use **constructor and super calls** for common employee information.

- Calculate and display **individual salaries and total payroll** efficiently.

❖ PROBLEM – III

AIM : (For Advanced Learners – home) Model a few media types that each compute a late fee differently and total the fees for a returned batch. Design the hierarchy and rules yourself.

CODE :

```
1  abstract class Media
2  {
3      String title;
4      int lateDays;
5      Media(String title,int lateDays)
6      {
7          this.title=title;
8          this.lateDays=lateDays;
9      }
10     abstract double lateFee();
11 }
12 class Movie extends Media
13 {
14     Movie(String title,int lateDays)
15     {
16         super(title,lateDays);
17     }
18     double lateFee()
19     {
20         return lateDays*5;
21     }
22 }
23 class CD extends Media
24 {
25     CD(String title, int lateDays)
26     {
27         super(title,lateDays);
28     }
29     double lateFee() {
30         return lateDays*3;
31     }
32 }
33 class Book extends Media
34 {
35     Book(String title,int lateDays)
36     {
37         super(title,lateDays);
38     }
39     double lateFee()
40     {
41         return lateDays*2;
42     }
43 }
```

```
44 public class MediaFee
45 {
    Run | Debug
46 public static void main(String[] args)
47 {
48     Media[] media={
49         new Movie(title: "Avengers",lateDays: 3),
50         new CD(title: "Music CD",lateDays: 4),
51         new Book(title: "Java Book",lateDays: 5)
52     };
53     double total=0;
54     for(Media m:media)
55     {
56         double fee=m.lateFee();
57         System.out.println(m.title+"Late Fee: "+fee);
58         total+=fee;
59     }
60     System.out.println("Total Late Fee: "+total);
61 }
62 }
```

OUTPUT :

```
AvengersLate Fee: 15.0
Music CDLate Fee: 12.0
Java BookLate Fee: 10.0
Total Late Fee: 37.0
PS C:\Users\TIRTH V PATEL\De
```

LEARNING OUTCOME :

- Design a **class hierarchy** for different types of media.
- Apply **abstraction, inheritance, and polymorphism** to real-world problems.
- Implement **different fee calculation rules** for different media types.
- Use **method overriding** to provide type-specific behavior.
- Calculate the **total fee for a batch of returned media** using object-oriented programming principles.